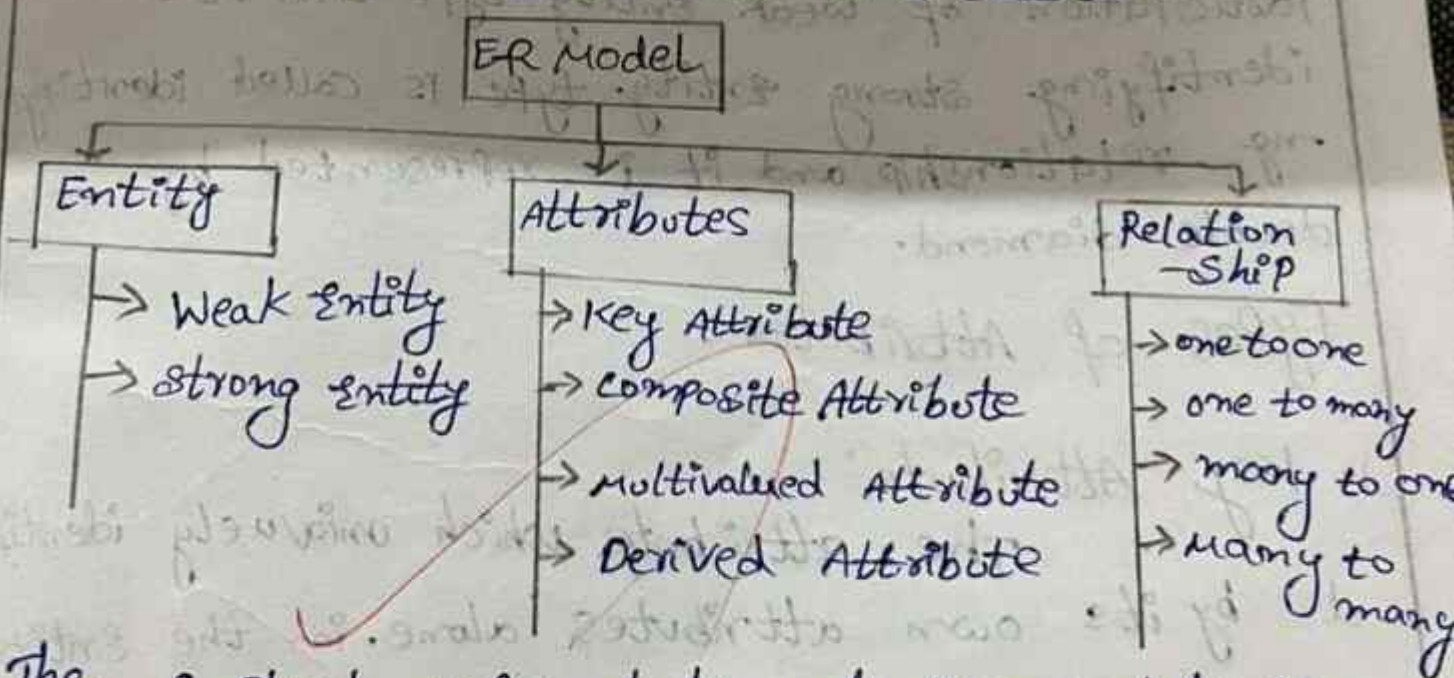


What is E-R Model?

The Entity - Relationship model (ER model) is a conceptual model for designing a databases. This model represents the logical structure of a database, including entities, their attributes and relationships between them.

- Entity: An object that is stored as data such as student course or company.
- Attributes: Properties that describes an entity such as "a student enrolls in a case."
- Relationship: A Connection between entities such as "a student enrolls in a course".



The graphical representation of this model is called Entity - Relation Diagram (ERD)

Entity:- An entity represents a real world Project, concept or thing about which data is stored in a database. It act as building block

of the database. Tables in relational database represent these entities.

1. Strong Entity:

A strong Entity is a type of Entity that has a key attributes that can uniquely identify represented by a rectangle in an E-R diagram.

2. Weak Entity:

A weak entity cannot be uniquely identified by its own attributes alone. It depends on a strong entity types and to be identified. The participation of weak entity type and its identifying strong entity type is called identifying relationship and it is represented by double diamond.

Types of Attributes:

1. Key Attribute:

The attribute which uniquely identifies by its own attributes alone in the entity set is called key attribute. ex: Roll No

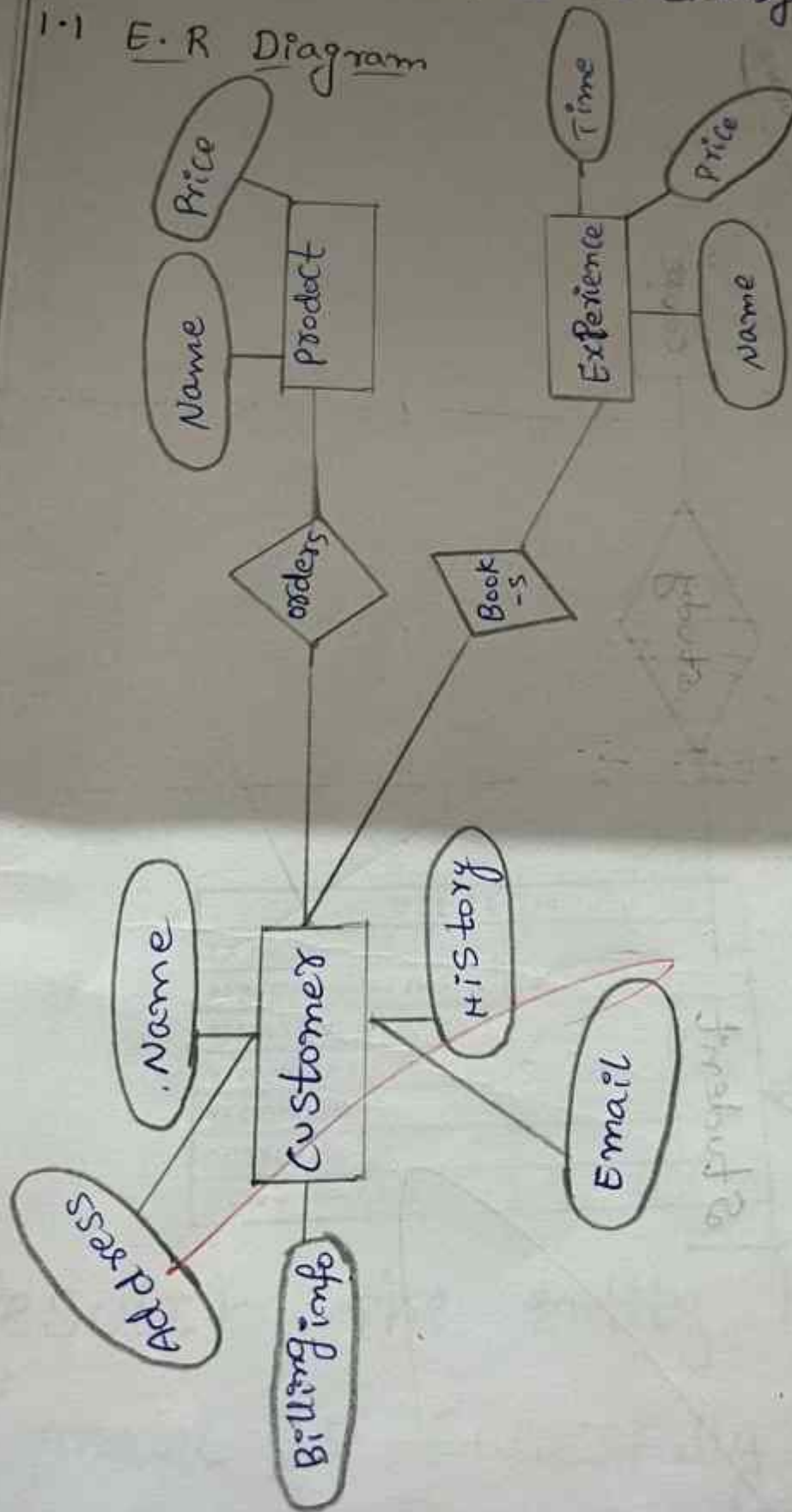
2. Composite Attribute

A attribute composed many other attributes is called a composite attribute.

Phone numbers

- Double Rectangle! Represents weak entity, which depend on entity

1.1 E-R Diagram

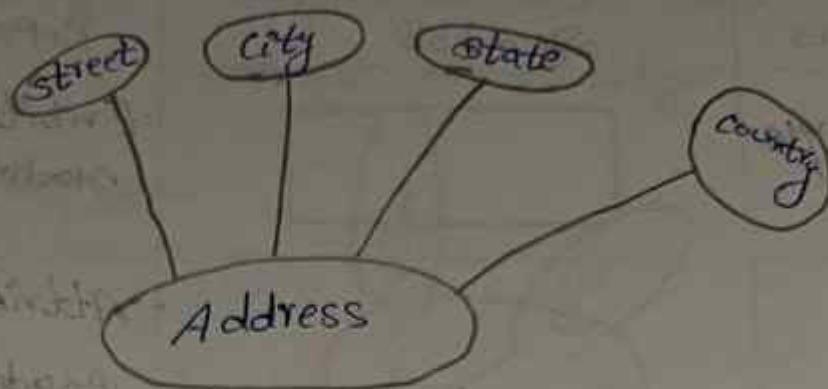


Entity - Relationship Model

Figures	Symbols	Represents
Rectangle		Entities in Model
Ellipse		Attributes in Model
Diamond		Relationships among Entities
Line		Attributes to entities and entity sets with other relationship types
Double ellipse		Multivalued attributes
Double rectangle		Weak Entity

- Rectangles: Rectangles represent Entities in the E R Model.
- Ellipses: Represents attributes in the E-R Model
- Diamond: Represents represents among entities
- Lines: Represents relation attributes to entity sets with other relationship
- Double Ellipse: Represents multi-valued Attributes such as student's multiple

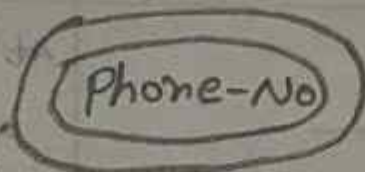
ex:



3. Multivalued Attributes :-

An attribute consisting of more than one value for a given entity.

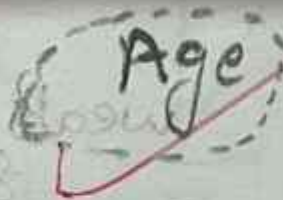
ex:



4. Derived Attributes :-

An attribute that can be derived from other attributes of the entity type is known as a derived attribute.

ex:



- TRUNCATE: Remove all records from a table including all spaces allocated for the records are removed

Syntax:

TRUNCATE TABLE table-name;

- COMMENT: Add comments to the data dictionary

Syntax:

COMMENT 'comment-text' ON TABLE
table-name;

RENAME: Rename an object existing in the database;

Syntax:

RENAME TABLE old-table-name TO new
- table-name

Example:

create table instructor (

ID char(6),

name varchar(20),

dept-name varchar(20),

Salary numeric(8,2)

DDL, DML and Integrity constraints Date:

Task 2.1

DDL commands:

DDL stands for data definition language in database management systems are used to define, modify, and manage the structure of a database and its objects. These commands do not directly manipulate the data within the tables but rather the schema or blue print of the database.

The Primary DDL Commands include:

- CREATE: create is used to new database object such as tables, views, indexes or databases themselves

Syntax:

create table table-name (column1 data-type, column 2)

DROP: used to delete objects from database

Syntax:

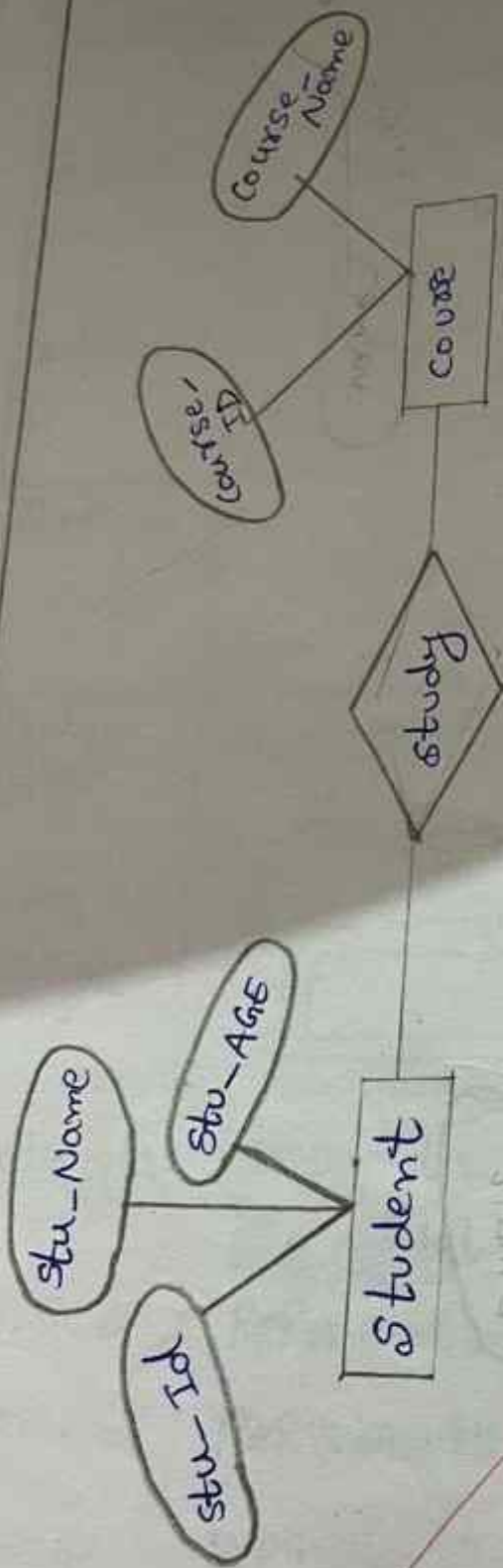
DROP TABLE table-name

ALTER: used to alter the structure of the database

Syntax:

ALTER TABLE table-name ADD column
column-name data-type;

1.2 - E-R Diagram using tool:



Oracle SQL code :-

save
copy
edit

```
CREATE TABLE Department (Dept ID  
Number(5), DeptCode  
Varchar2(10) UNIQUE);
```

Explanation :- No two departments can have the same DeptCode. It helps maintain data unique in columns like email, username etc.

3. PRIMARY KEY constraint :

The PRIMARY KEY constraint uniquely identifies each record in a table. It is a combination of NOT NULL and UNIQUE. A table can have only one primary key.

Oracle SQL code :-

save
copy
edit

```
CREATE TABLE student (  
Student ID numbers(5) PRIMARY KEY  
NAME varchar(50));
```

DDL with Integrity Constraints

1. NOT NULL CONSTRAINT:

The NOT NULL constraint ensures that a column cannot contain NULL values. It enforces the rule that every row must have a value in this column.

SQL Code

SQL

COPY

EDIT

```
CREATE TABLE Employee (Emp-ID number(5),
                          Name varchar2(50) NOT NULL);
```

Explanation:

The name column must always have value. If you try to insert a row without a name, Oracle will throw an error.

2- UNIQUE Constraint:

The UNIQUE constraint ensures that all values in a column are different. It allows NULL values, but only one if the column has a single UNIQUE constraint.

Task-2.2

DML Commands:

Data manipulation language is used to manage data inside tables.

Common DML Commands are:

Insert:- used to add new data

Syntax:-

```
INSERT INTO student (student ID, NAME  
Age)
```

SELECT:- used to retrieve the data

Syntax:-

```
SELECT * FROM student;
```

Update:- used to modify the existing data

Syntax:-

```
UPDATE student
```

```
SET Age = 20
```

DELETE:- used to delete specific data

Syntax:-

```
DELETE FROM student
```

Explanation!

If status is not specified while inserting a row, oracle will automatically insert 'pending'.

VEL TECH	
EX No.	
PERFORMANCE (3)	
RESULT AND ANALYSIS (3)	
VIVA VOCE (3)	
RECORD (3)	
TOTAL (30)	
MARKS WITH DATE	

Result Thus the SQL relations using commands has been completed successfully

5. CHECK constraint!

The CHECK constraint limits the values that can be inserted into a column. It ensures data follows specific rules.

oracle SQL code!

SQL

Copy

Edit

```
CREATE TABLE product (product ID number  
(5), price number (8,2) CHECK  
(price > 0));
```

Explanation:- only positive values are allowed for price. Negative or zero values are not allowed for price. Negative or zero values will cause an error.

6. DEFAULT constraint:-

The default constraint assigns a different default value to a column if no value is provided during insertion.

oracle SQL code:-

SQL

Copy

Edit

```
CREATE TABLE orders (order ID number (5),  
status varchar (20) DEFAULT Pending);
```

4. FOREIGN KEY constraints:

The FOREIGN KEY constraint is used to link two tables. It enforces a relationship between the foreign key column and the primary key in another table.

oracle SQL Code:

SQL

Copy

Edit

-- main table

```
CREATE TABLE course (course ID number(5)  
PRIMARY KEY, course Name  
Varchar2(50));
```

-- dependent table

```
CREATE TABLE Enrollment (  
enrollment ID number(5) PRIMARY KEY,  
course ID number(5),  
FOREIGN KEY ((course ID) REFERENCES  
course (course ID));
```

Explanation:- The course ID in enrollment must exist in the course table. You cannot insert in void course ID.

Sample queries and output:-

1. Retrieve member name and with letter 'n' & member no between 111 & 115

Query:
SELECT first_name, last_name, salary FROM
Employees WHERE first_name LIKE '%n%';

Output:-

name
John
Ben
Anna

2. List the books whose page count b/w 700 & 800 between clause, and operator

Query:-
SELECT * from book where page count b/w 700 & 800;
Output:-

isbn	title	authors	page count
1111	Operating Systems	Siberchatz	750

3. Find the records who has minimum-number pagecount - Aggregate.

Query: select min(pagecount) from book

Output:-
min (Page count)
330

4. Find the records whose issue date greater than or equal to 07-02-2023

Query: select * from borrowed where issue date
>= '2023-02-07';

Output:-

memb-no	isbn	issued date
113	1110	2023-02-07

5. List the memb-no, but the same memb-no listed ones

Query:

select distinct memb-no from member;

Syntax: SQL > UPDATE relation name SET field-name1 = data, field-name2 = data, WHERE field-name = data;

Ex: SQL > UPDATE member SET Sname = 'Smit' WHERE Sno = 11

3. DELETE-FROM: This is used to delete the records of a relation but it will retain the structure of that relation.

a) DELETE-FROM: This is used to delete all the records of a relation.

Syntax: SQL > Delete from relation name

Ex: SQL > ~~DE~~ Delete from std.

b. Delete-from-where: This is used to delete record from a relation

Syntax: SQL > Delete from relation name

Condition-

Ex: SQL > Delete from relation-name where Can student where Sno = 15;

4. Truncate:- This command will remove the data permanently but structure will not be removed

Syntax: Truncate table < table names

Ex: Truncate table student;

Task - 3

Using clauses Operations and

Functions in queries

Aim:- Implementation of DML Commands using clauses, operators and functions in queries

Data Manipulation Language (DML):- The data manipulation language (DML) is used to retrieve, insert and modify database information. These commands will be used by all database users during the routine operation of the database. take a brief look at the basic DML Commands.

1. INSERT 2. UPDATE 3. DELETE

1. INSERT INTO: This is used to add records into a relation

There are three types of INSERT INTO where as.

Syntax:- INSERT INTO <relation/table name> (field-1, field-2, ..., field-n) VALUES (data-1, data-2, ..., data-n);

Ex:- SQL> INSERT INTO student VALUES ('VTU29283', 'preethi', 'CSE', 'Female');

2. UPDATE SET-WHERE: This is used to update the content of a record in a relation

Aggregate functions (Multi-Row operation)

Aim: To study and implement aggregate functions (count(), sum(), Avg(), Min(), max()) on a sample student Database.

Procedure:-

1. Create a table named students
2. Insert sample records
3. write queries using aggregate functions
4. observe and record the output.

commands with explanation:-

1. count the total numbers of student.
 select count (*) as total-students from students

2. Find the highest marks obtained by a student.
 select MAX(marks) as highest mark from students;

3. Find the average marks of the students
 select Avg (marks) as average - mark from students;

4. Find the minimum marks among student in the ECE

select min(marks) as min-Ece-mark from students where department 'ECE';

output:-

1. It will return one row with one column
Total_Students
 n

EX:- 1) If there are 5 rows $\rightarrow$ output
 2) If there are 0 rows $\rightarrow$ output = 0

2) output:- If the marks column has values:
 65, 72, 88, 90, 76 $\Rightarrow$ Highest - mark
 90

3) output:- Average = $\frac{65+72+88+90+76}{5} = 78.2$
average - mark
 78.2

4) ~~output~~:- output:-

min - ECE - mark
 65

name	department	marks
A	ECE	72
B	ECE	65
C	ECE	80
D	CSE	75

5) output:-

name	department	marks
A	CSE	80
B	CSE	75
C	ECE	70
D	ECE	85
E	MECH	60

6) output:-

1. Average for each department

• CSE $\rightarrow (80+70)/2 = 75$

• ECE $\rightarrow (90+60)/2 = 75$

• mech $\rightarrow 75$

2. Sorted (Desc)

department	avg - marks
CSE	75
ECE	75
Mech	75

output:-

Membr-no
111
113
114
115
116

6. Combine the records of member and book table
- union

query: select memb-no From member UNION
SELECT memb-no
FROM borrowed;

output:-

Membr-no
111
113
114
115
116
117
121

7. Group by the number based on their gender & department

query:- SELECT memb-no FROM member Group by
gender, department

output:-

Membr-no
111
111
116
117

8. Find the authors & their publication details using group by and order by clauses

query:- select author, Publisher, Count (*) no
From book group by authors, Publisher order by
authors, Publisher order by authors

output:-

authors	Publisher
Mark allen	Springer
silberchatz	mc grow
silberchatz	Springer

Task-4

Developing queries with DML multi-functions and operators

Perform the advanced query processing (multi row abilities) and test its heuristic using the designing of optimal co-re and nested subqueries such as finding summary statistics

Consider the schema for

Employ.

EMPLOYEES (emp-no, emp-name, dept, dept, salary, age)

orders (emp-no, order id, price, qty-ord, item id, item name, qty-hand, qty-ord, item rate)

Queries using union, Intersect, Minus:

union!- The union operator returns all distinct rows selected by two or more queries

SQL > select emp-no from employees;

Output!

SQL > select emp-no from employees union

Select count (*) as total 8 students from Students.

Select count (*) as students in 10A from student where S-class = '10A';

Select sum (marks) as total students from student;

Select sum (marks) as marks in 10B

from student where S-class = '10B';

Select Avg (marks) as average marks from student;

Select S-class, Avg (marks) as Avg marks from student group by S-class;

Select min (marks) as lowest marks from student;

Select min (age) as youngest age from student;

VEL TECH	
EX No.	3
PERFORMANCE (5)	4
RESULT AND ANALYSIS (5)	4
VIVA VOCE (5)	3
RECORD (5)	1
TOTAL (20)	11
ICA MARKS	10

Result:- The task to implement the DML Commands are executed successfully.

5. Find the total marks scored by students in each department

Select department, sum(marks) as total marks from students group by department

6. Find the average marks per department ordered by average marks descending

Select department, Avg(marks) As Avg-marks from students

Group by department

order by Avg-marks DESC;

Create table student (s-ID int(10) Primary key, s-name char(20), s-class varchar(20), s-marks int, s-Age int);

Insert into student values (1, 'Pavon', '10A', 85, 15);

Insert into student values (2, 'Raj', '10B', 92, 14);

Insert into student values (3, 'Singh', '10A', 78, 15);

Insert into student values (4, 'Priya', '10B', 86, 16);

Insert into student values (5, 'Neha', '10A', 65, 14);

Insert into student values (6, 'Teju', '10A', 90, 15);

Insert into student values (7, 'Rishi', '10C', 55, 16);

Insert into student values (8, 'Ravi', '10B', 70, 15);

Output:-

SQL > select emp no, emp name, salary from employees order by salary desc;

Output:-

SQL *plus having following operations

SQL > select salary + comm from emp-master
salary + comm

Output :-

SQL > select 12 * (salary + comm) annual-net
sal from emp-master;

Subqueries :-

SQL > select * from employees

SQL > insert into employees select * from
employees where emp-id in (select emp.
from employees);

SQL > update employees set salary = salo
where department in (select department
employer whose department = 'Sales');

Delete from employees where depart
in (select department from employees
department = 'Sales');

output:-

Group-by-having:-

The having clause was added to SQL because the where keyword could not be used with aggregate functions. The having clause must follow the group by clause in a query and must also precede the order by clause if used.

SQL > select dept no - Count (*) from emp
group by dept no having dept no is not

output:-

ORDER BY:-

This query is used to display a selected set of fields from a relation in an ordered manner based on name field.

Syntax:-

select < column(s) > from < Table name >
where [condition(s)] [order by < column name
[asc / desc],

SQL > select emp no, emp name, salary from
employees, order by salary.

Select emp_no from orders;

Union all:-

SQL > select emp_no from employees
union all select emp_no from orders;

Intersect:-

SQL > select emp_no from employees interse_{ct}
Select emp_no from orders;

Minus:-

SQL > select emp_no from employees minus
Select emp_no from orders;

Queries using Group by having clause
and order clause:-

Group by:-

This query is used to group to
the records in a relation together for
each and every value of a specific key
and then display them for a selected set
of fields the relation

SQL > select dept no, Count (*) from emp
group by dept no;

Increase salary of employees in the IT department

```
UPDATE employee SET Salary = Salary + 5000  
WHERE Dept-ID = (select Dept-ID FROM Department WHERE Dept-name = 'IT');
```

VELTECH	
EX No.	
PERFORMANCE (5)	
RESULT AND ANALYSIS (5)	
VIVA VOCE (5)	
RECORD (5)	
TOTAL (20)	
SIGNATURE	

Result:- Thus, the development of queries with SQL - multirows function & operations has been completed successfully.

```
INSERT INTO (student-ID, name, Graduation-year)
SELECT student-ID, name, Passout-Year
FROM student
```

```
WHERE Passout-Year < 2023;
```

```
INSERT INTO Employee-Back-up (emp-ID, emp-
name, Department)
```

```
VALUES (
```

```
(SELECT emp-ID FROM employee WHERE
emp-name = 'NANI');
```

```
(SELECT emp-name FROM employee WHERE
emp-name = 'Arjun');
```

```
(SELECT Department from employee WHERE
emp-name = 'NANI');
```

```
DELETE, FROM student WHERE student-ID
IN (SELECT student-ID FROM ALUMNI WHERE
graduation-Year < 2023);
```

```
DELETE Employees not assigned to
any department
```

```
DELETE FROM employee WHERE Dept-ID
NOT IN (SELECT Dept-ID from department
```


IN:-

Query:- SELECT * FROM employees where department in ('sales', 'marketing');

NOT IN:-

Query:- SELECT * FROM employees where department NOT IN ('sales', 'marketing');

Exists:-

Query:- SELECT * FROM employees where (select * from orders where orders emp_no = (link unavailable)

NOT EXISTS:-

Query:- SELECT * FROM employee where not exist (SELECT * FROM orders where orders emp_no = (link unavailable));

ALL:-

Query:- SELECT * FROM employees WHERE salary > ALL (select salary from employees WHERE department = 'sales');

ANY:-

Query:- SELECT * FROM employees WHERE salary > (SELECT salary from employees WHERE department = 'sales');

clause. since the where clause uses the Comparison operator (=) to perform a join

It is said to be equi-join. It combines the matched rows of tables. It can be used as follows:-

- To insert records in the target table
- To create tables and insert records in this table
- To update records in the target table
- To update records in the target table
- To create views

non equi-join:-

It specifies the relationship between columns belonging to different tables by making use of relational operators other than '='.

Example:-

```
select * from item cust where item-id < cust
```

Table Aliases:-

Table Aliases are used to make multiple table queries 'shorter and more readable'. we give alias name to the table in the 'from clause' and use it instead of the name throughout the query.

SYNTAX:-

```
SELECT COLUMN 1. Column 2. Column  
3. FROM table - name 1, table - name 2  
WHERE table - name 1 - Column name = table  
name 2. Column name;
```

Types of Joins:-

1. simple join
2. self join
3. outer join

1. simple join:-

It is the most common type of join. It return the rows from 2 tables having a common column and is further classified into

Equality Join:-

A join, which is based on equalities, is called Equality Join.

Example:-

```
select * from item, cust where item.id
```

In the above statement, $\text{item.id} = \text{cust.id}$,
Performs the join statement. It retrieves
from both the tables provided they both
have the same id as specified by the where

Task - 5

writing Join queries, equivalent AND/OR recursive queries

Title:- Implementation of different types of joins and recursive queries

- A SQL Join Combines records from two tables
- A query locates related column values in the two tables
- **INNER JOIN** is the same as **JOIN**: the keyword

INNER is optional

Objective:- To implement different types of Joins and recursive queries

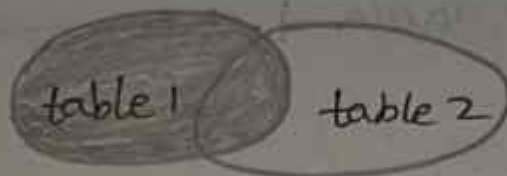
Theory:-

The SQL joins clause is used to combine records from two or more tables in a database

A JOIN is a means for combining fields from two tables using values common to each. The join is actually performed by the 'WHERE' clause which combines specified rows of tables

FULL OUTER JOIN table 2 on table 1.
 Column-name = table 2 - Column-name;

LEFT JOIN



RIGHT JOIN



INNER JOIN



FULL OUTER JOIN



Display the employee details, departments that the departments are same in both the employees.

Code:

```
select e_ID, e_first name, e last name, e_job_ID, e_dept_name
from employees e inner join departments
d on e_dept_ID = d.dept_ID;
```

output:

Employee_ID	First name	last name
101	Neena	Kochhar
102	Lex	De Haan
103	Alexander	Hunold
104	Bruce	Ernst
105	David	Austin

(INNER JOIN): Returns records that have matching values in both tables.

SELECT Column-name(s) FROM table 1 INNER JOIN Table 2 ON Table 1.Column-name=table2.Column-name;

Left (OUTER) JOIN: Return all records from the left table, and the matched records from the right table.

SELECT Column-name(s) FROM table LEFT JOIN table 2 ON table 1.Column-name=table2.Column-name;

Right (OUTER) JOIN:- Return all records from the right table, and the matched records from the left table

SELECT Column-name(s) FROM table RIGHT JOIN table 2 ON table 1.Column-name=table2.Column-name;

Full (OUTER) JOIN:- Return all records when there is a match in either left right table.

SELECT Column-name(s) FROM table

2) Self join:-

Joining of a table to itself is known as Self join.

It joins one row in a table to another.

It can compare each row of the table itself and also with other rows of the same table.

Ex:-

```
select * from emp x, emp y where x.salary >
```

```
(select avg(salary) from emp where x.dept = y.dept no);
```

3) outer join:-

It extends the result of a simple join.

An outer join returns all the rows returned by simple join as well as those rows from one table that do not match any row from the table. The symbol (+) represents outer join.

Different types of SQL joins.

Here are the different types of the join in SQL >

1
 0
 1
 2
 3
 5
 8
 13
 21
 34

(10 rows)

VEL TECH	
EX No.	
PERFORMANCE (5)	
RESULT AND ANALYSIS (5)	
VIVA VOCE (5)	
RECORD (5)	
TOTAL (70)	
DATE WITH DATE	

Result:- Thus the join queries and recursive queries has been successfully completed.

output:
Employee-ID
101
102

first-name
Neena
Lex

last-name
Kochhar
DeHaan

f1
0
1
2
3
5
8
13
21
34
(1

Recursive queries:

Syntax:

```
with recursive [cte-name] (column, --) AS (
  [non-recursive-terms]
  union all
  [recursive-term])
select -- from [cte-name];
```

+ write a recursive query to create a multiplication table by 2

```
with Recursive v2 (result) AS /
```

```
select 1
```

```
union all
```

```
select result * 2 from v2
```

```
select * from v2 limit 10;
```

output

result

1

2

4

8

16

32

64

128

256

512

1024

2048

4096

8192

16384

32768

65536

131072

262144

524288

1048576

2097152

4194304

8388608

16777216

33554432

67108864

134217728

268435456

536870912

1073741824

2147483648

4294967296

8589934592

17179869184

34359738368

68719476736

137438953472

274877906944

549755813888

1099511627776

2199023255552

4398046511104

8796093022208

17592186044416

35184372088832

70368744177664

140737488355328

281474976710656

562949953421312

1125899906842624

2251799813685248

4503599627370496

9007199254740992

18014398509481984

36028797018963968

72057594037927936

144115188075855872

288230376151711744

576460752303423488

1152921504606846976

2305843009213693952

4611686018427387904

9223372036854775808

18446744073709551616

36893488147419103232

73786976294838206464

147573952589676412928

295147905179352825856

590295810358705651712

1180591620717411303424

2361183241434822606848

4722366482869645213696

9444732965739290427392

18889465931478580854784

37778931862957161709568

75557863725914323419136

151115727451828646838272

302231454903657293676544

604462909807314587353088

1208925819614629174706176

2417851639229258349412352

4835703278458516698824704

9671406556917033397649408

19342813113834066795298816

38685626227668133590597632

77371252455336267181195264

154742504910672534362390528

309485009821345068724781056

618970019642690137449562112

1237940039285380274899124224

2475880078570760549798248448

4951760157141521099596496896

9903520314283042199192993792

19807040628566084398385987584

39614081257132168796771975168

79228162514264337593543950336

158456325028528675187087900672

316912650057057350374175801344

633825300114114700748351602688

1267650600228229401496703205376

2535301200456458802993406410752

5070602400912917605986812821504

10141204801825835211973625643008

20282409603651670423947251286016

40564819207303340847894502572032

81129638414606681695789005144064

162259276829213363391578010288128

324518553658426726783156020576256

649037107316853453566312041152512

1298074214633706907132624082305024

2596148429267413814265248164610048

5192296858534827628530496329220096

10384593717069655257060992658440192

20769187434139310514121985316880384

41538374868278621028243970633760768

83076749736557242056487941267521536

166153499473114484112975882535043072

332306998946228968225951765070086144

664613997892457936451903530140172288

1329227995784915872903807060280344576

2658455991569831745807614120560689152

5316911983139663491615228241121378304

10633823966279326983230456482242756608

21267647932558653966460912964485513216

42535295865117307932921825928971026432

85070591730234615865843651857942052864

170141183460469231731687303715884105728

340282366920938463463374607431768211456

680564733841876926926749214863536422912

1361129467683753853853498429727072845824

2722258935367507707706996859454145691648

5444517870735015415413993718908291383296

10889035741470030830827987437816582766592

21778071482940061661655974875633165533184

43556142965880123323311949751266331066368

87112285931760246646623899502532662132736

174224571863520493293247799005065324265472

348449143727040986586495598010130648530944

696898287454081973172991196020261297061888

1393796574908163946345982392040522594123776

2787593149816327892691964784081045188247552

5575186299632655785383929568162090376495104

11150372599265311570767859136324180752990208

22300745198530623141535718272648361505980416

44601490397061246283071436545296723011960832

89202980794122492566142873090593446023921664

178405961588244985132285746181186892047843328

356811923176489970264571492362373784095686656

713623846352979940529142984724747568191373312

1427247692705959881058285969449495136382746624

2854495385411919762116571938898990272765493248

5708990770823839524233143877797980545530986496

11417981541647679048466287755595961091061972992

22835963083295358096932575511191922182123945984

45671926166590716193865151022383844364247891968

91343852333181432387730302044767688728495783936

182687704666362864775460604089535377456991567872

365375409332725729550921208179070754913983135744

730750818665451459101842416358141509827966271488

1461501637330902918203684832716283019655932542976

2923003274661805836407369665432566039311865085952

5846006549323611672814739330865132078623730171904

11692013098647223345629478661730264157247460343808

23384026197294446691258957323460528314494920687616

46768052394588893382517914646921056628989841375232

93536104789177786765035829293842113257979682750464

187072209578355573530071658587684226515959365500928

374144419156711147060143317175368453031918731001856

748288838313422294120286634350736906063837462003712

1496577676626844588240573268701473812127674924007424

2993155353253689176481146537402947624255349848014848

5986310706507378352962293074805895248510699696029696

11972621413014756705924586149611790497021399392059392

23945242826029513411849172299223580994042798784118784

47890485652059026823698344598447161988085597568237568

95780971304118053647396689196894323976171195136475136

191561942608236107294793378393788647952342390272950272

383123885216472214589586756787577295904684780545900544

76

Display the Employee name & Dept name implementing a left outer join.

```
select e-first-name || ' ' || e-last-name as employee  
d-dept-name from employees left outer join depart-  
ments d on e-dept-ID = d-dept-ID;
```

output:-

Employee - Name

Department name

Neena Kochhar

Administration

Alex De Haan

Marketing

Alexander Russell

IT

Bruce Ernst

IT

David Austin

Purchasing

John Smith

NULL

Display the employee name and department name by implementing a right outer join

```
select e-first-name || ' ' || e-last-name as employee  
d-dept-name from employees right outer join  
departments d on e-dept-ID = d-dept-ID;
```

output:-

Employee name

Department name

Neena Kochhar

Administration

Alex De Haan

Marketing

Alexander Russell

IT

Bruce Ernst

IT

David Austin

Purchasing

NULL

Finance

d) Display the details of those who draw the greater than the average salary

```
select employee-ID, first-name, last-name  
salary, dept-ID from employees where sal  
(select Avg(salary) from employees);
```


Output:-

Total Players : 14

Total Age : 342

Average Age : 24.42

Write a PL/SQL block that inserts a new player record into the player table

DECLARE

V-PlayerID Varchar(6) := '&PlayerID';

-- You can generate a unique

V-TeamID Varchar(6) := '&TeamID';

-- Replace with the actual Team ID

V-FNAME Varchar(30) := '&FName';

V-L Name Varchar(30) := '&L Name';

V-Age Number(5,2) := '&Age';

V-Date of Birth date := To-Date('&DOB/yy

-- Replace with the actual Date of Birth -MM-DD')

V-Playing Role Varchar(25) := '&Playing R

V-email Varchar(40) := '&Email';

V-contact - no Number := @Phone;

-- Replace with the actual contact no

BEGIN

BEGIN

-- using a cursor to loop through
Players

FOR Player-REC IN (select Age FROM
Player) LOOP

total-age := total-age + Player-REC.Age;

-- Summing up the ages

num-Players := num-Players + 1;

-- counting the number of Player

END LOOP;

-- Calculating the average age

IF num-Players > 0 THEN

avg-age := total-age / num-Players;

END IF;

-- Displaying the result

DBMS-output.put-line ('Total Players: '
//num-Players);

DBMS-output.put-line ('Total-Age: '//
total-age);

DBMS-output.put-line ('Average Age: '//
avg-age);

Task-6

Procedures, Functions and Loops

Aim:- To write a programming using PL/SQL Procedure, Function and Loop on number theory and business scenarios like.

1. Write a PL/SQL block that calculate the average age of players and displays the result.
2. Write a PL/SQL block that inserts a new player record into the Player table.
3. To create a function that returns the total number of teams in a particular cricket board
4. To write a non-recursive PL/SQL procedure to retrieve even-numbered Player IDs registered for any tournament

Write a PL/SQL block that calculates the average age of players and display the result.

~~DECLARE~~

~~total_age NUMBER := 0;~~

~~num_players NUMBER = 0;~~

~~avg_age NUMBER = 0;~~

```

SQL>
Declare
number res;
Begin
res := Get Total Terms In Board
      (B7DOT)
DBMS_output.put_line ("No. of Teams: " || res);
END;
/
no. of teams = 2

```

To Create a non-recursive PL/SQL Procedure to retrieve even-numbered Player IDs registered for any tournament.

create or replace Procedure Get Even Numbered Player IDs is

Begin

For Player_rec IN (select PlayerID
From Players where to-number
(PlayerID) mod 2 = 0)

Loop

DBMS_output.put_line ("Even-Numbered
Player ID: " || Player_rec.PlayerID);

Enter the contact-no : 9988776655

Player record inserted successfully

To create a function that returns total number of teams in a particular cricket Board.

create or Replace Function Get Total
In Board (Board ID varchar 2)

Return number is V-Total Teams
BEGIN

select count (*) INTO V-Total
From Team where Board ID = Board ID
Return V-Total Teams;

Exception

when No-Data-Found. Then

-- Handle the case when the
doesn't

Return -1; -- Return a negative
to indicate an error

END Get Total Teams I
/
Function successfully created

```
INSERT into Player (Player ID, Team ID,  
F Name, Age, Date of Birth, Playing Role,  
Email, Contact-no)  
Values (V-Player ID, V-TeamID, V-FName,  
V-LName, V-Age, V-Date of Birth,  
V-Playing Role, V-email, V-Contact-no),  
COMMIT;
```

```
DBMS_output.Put_Line("Player record  
inserted successfully")
```

```
Exception
```

```
when others Then
```

```
DBMS_output.Put_Line('Error: '|| SQLERRM)
```

```
'ROLL BACK;
```

```
END;
```

```
/
```

Enter the Player ID: 676

Enter the Team ID: ICCB01

Enter the F Name: Rahul

Enter the L Name: Sharma

Enter the Age: 23

Task-7

Triggers, Views and Exceptions

Aim:

To conduct events, views, exceptions for
crud operations for restricting them.

a) To create a trigger in PL/SQL that
automatically inserts a new record in
b) To create a view that displays the
details of players along with their
details

c) To write a non-recursive PL/SQL
-ure to retrieve even-numbered players
registered for any tournament.

To create a trigger in PL/SQL that
automatically inserts a new record in the
match-result table when a new record
is inserted into the match table.

create or replace trigger insert-match
After insert on match
For each row

Begin

Insert into match-result (matchID, TeamID,
Result)
VALUES (:new-match ID, :new-Team ID,
'Pending');

```

END LOOP;
END Get Even Numbered PlayerIDs;
/

```

VELTECH	
EX No.	6
PERFORMANCE(S)	4
RESULT AND ANALYSIS(S)	4
VIVA VOCE(S)	4
RECORD(S)	2
TOTAL(S)	18
VIVA WITH DAT	20

Result:- Thus the PL/SQL Procedure functions and loops on number theory and business scenarios experiment was successfully completed and results are verified.

2) output:-

Player Details:-

Player ID : 676

Team ID : CCBOI

First Name : Rahul

Last Name : Sharma

Age : 23

Date of Birth : 17-07-1999

Playing Role : All Rounder

Email : rahulsharma@gmail.com

Contact no : 9797181815

3) output:-

No. of teams : ~~ID~~ 2

Player ID :

101

102

103

104

105

4) output:-

Even-numbered Player ID : 102

Even-numbered Player ID : 104

1) output:-

Total Players : 14

Total Age : 342

Average Age : 24.42

To write a non-recursive PL/SQL
Procedure to retrieve even-numbered
Player IDs registered for any tourna-
-ment.

create or replace procedure get
employer IDs for

Tournament (in - tournament-id
number, out, even - player-ids

sys: OP(IN NUMBER LIST) AS

BEGIN

out-even-player-ids := sys.OPCINUMBER
LIST);

-- Initialise the collection

-- populate the collection with
even-numbered Player ID

out-even-player-ids.extend;

out-even-player-ids(out-even-player-
ids.count) := Player-rec.Player ID;

END LOOP;

END;

/

DECLARE

tournament-id number := 1,2,3;

-- Replace with the desired tourname-
-nt ID.

Values (: New-empid / sysdate);
END;

/

To create a view that displays the details of players along with their team details.

SQL > create View Player Team Details
AS select P.PlayerID, PF Name AS
Player Name, P-team ID, P.Coach AS
Player Coach, P.Captain AS Player Captain,
(select t.name from team t where
t.teamID = P.teamID) AS TeamName,
(select t.Coach from team where t.team
ID = P.team ID) AS Team Coach,
(select t.Captain from team t where
t.team ID = P.team ID) AS Team
Captain from Player P;

SQL > select * from Player TeamDetails

-- Assuming a default value of 'pending'
for result
END;

/

Syntax of a trigger:

create or replace trigger trigger_name
Before/After Insert (update) delete
on table_name

[for each row] --- row-level-trigger

Begin ... trigger logic

End;

/

Trigger:

create table employees (emp-id int,
emp-name char(10));

create table employee_log (emp-id int,
log-date date);

create or replace trigger trg-log-emp
- insert

After insert on employees

for each row

Begin

Insert into employee_log (emp-id, log-date)

Task-8

Normalizing Databases using functional dependencies upto BCNF

Upon relational tables created in task-2, Perform normalization upto BCNF based on given dependencies as following for the assumed relations specified below.

Employee Database:

1. Identify employee attributes: Employee-ID, Name, Department, Job-Title, Manager-ID, Hire-Date, Salary

2. Define relational Schema: Employee (Employee-ID, Name, Department, Job-Title, Manager-ID, Hire-Date, Salary)

3. Determine functional dependencies (FD's) between attributes:

- Employee-ID $\rightarrow$ Name, Department, Job-Title

Manager-ID, Hire-Date, Salary

- Department $\rightarrow$ Manager-ID

- Manager-ID $\rightarrow$ Name

a) output:-

Result ID	Match ID	Team ID	Result
1	M101	T01	Pending

b) output:-

Player ID	Player Name	Team ID	Player Coach	Player Captain
101	Rahul	T01	Mr. Sharma	Virat
102	Ramesh	T02	Mr. Verma	Rohit

Team Name	Team Coach	Team Captain
Titans	Mr. Sharma	Virat
Warriors	Mr. Verma	Rohit

c) output:-

If Player table has matching datas:-

Even Player ID : 102

Even Player ID : 106

Even Player ID : 110

VELTECH	
EX No.	7
PERFORMANCE (S)	4
RESULT AND ANALYSIS	6
VIVA VOCE (S)	3
RECORD (S)	1
TOTAL (20)	10
SIGN WITH DATE	

Result:- Thus the triggers, views & sequences experiment was successfully completed & results are verified

To write a non-recursive PL/SQL Procedure to retrieve even-numbered Player IDs registered for any tournament.

create or replace Procedure get
employee IDs for

Tournament (in - tournament-id
Number, out, even - player-ids

SYS: OP(IN NUMBER LIST) AS

BEGIN

out-even-player-ids := sys.opc(INNUMBER
LIST);

-- Initialise the collection

-- Populate the collection with
even-numbered Player ID

out-even-Player-ids.Extend;

out-even-Player-ids(out-even-Player-
ids.Count) := Player-rec.Player ID;

END LOOP;

END;

/

DECLARE

tournament-id Number := 1,23;

-- Replace with the desired tournament
-nt ID.

Example:- DEPARTMENT
CREATE TABLE (Department-ID VARCHAR2(100)
PRIMARY KEY, Manager-ID NUMBER,
Name VARCHAR2(100));

CREATE TABLE EMPLOYEE (Employee-ID
NUMBER PRIMARY KEY, Name VARCHAR
2(100), Department-ID VARCHAR2(100), Job
Title VARCHAR2(100), Hire-Date DATE,
Salary Number(10,2), CONSTRAINT
fk_dept FOREIGN Key (Department-ID
REFERENCES Department (Department-ID)).

STEP 4: Convert to 3NF

1) Ensure there are no transitive dependencies.

2) move non-key attributes to separate tables if they depend on another non-key attribute

- create Manager table: Manager (Manager-ID,
Name).

- update Department table: Department
(Department-ID, Manager-ID

Step 2: Convert to 1NF

1. Eliminate repeating groups or arrays
2. Create separate tables for each repeating group

Example :-

```
CREATE TABLE Employee (Employee-ID  
NUMBER PRIMARY KEY, Name VARCHAR  
(100), Department VARCHAR2(100), Job-Title  
VARCHAR2(100), Manager-ID NUMBER,  
Hire-Date DATE, Salary Number(10,2)).
```

Step 3: Convert to 2NF

Ensure each non-key attribute depends on the entire Primary Key.

Move non-key attributes to separate tables if they depend on only part of the Primary Key.

- create Department table: Department
(Department-ID, ~~Manager-ID~~, Name);

- create Employee table: Employee
(Employee-ID, Name, Department-ID,
Job-Title, Hire-Date salary).

Step 2: Convert to 1NF

1. Eliminate repeating groups or arrays
2. Create separate tables for each repeating group

Example :-

```
CREATE TABLE Employee (Employee-ID  
NUMBER PRIMARY KEY, Name VARCHAR2  
(100), Department VARCHAR2(100), Job-Title  
VARCHAR2(100), Manager-ID NUMBER,  
Hire-Date DATE, Salary Number(10,2)).
```

Step 3: Convert to 2NF

Ensure each non-key attribute depends on the entire primary key.

Move non-key attributes to separate tables if they depend on only part of the primary key.

- Create Department table: Department (Department-ID, ~~Manager-ID~~, Name);
- Create Employee table: Employee (Employee-ID, Name, Department-ID, Job-Title, Hire-Date, Salary).

Additional dependencies include:

- Book ID $\rightarrow$ Price
- Total Price $\rightarrow$ quantity $\times$ Price

Step 3: Normalize tables

1NF

- All tables already have atomic attributes
- Each field contains only one value

2NF:

- 2NF requires 1NF and no partial dependency on a composite key

• Since none of tables has composite primary keys this is mostly satisfied.

- If sales used (Book-ID, Customer ID, Sale Date) as a composite key, we'd check partial dependencies:

3NF:

- Remove transitive dependencies.

Look for transitive dependencies:

- In book, Publisher ID $\rightarrow$ Publisher Name, Publisher Address (Publisher info depends on Publisher ID, which is not the primary key in book.

- In book, Author ID $\rightarrow$ Author Name, Author Country.

- So, Publisher and Author info should be in their own tables.

Normalized Schema

1. Employee (Employee-ID, Name, Department-ID, Job-Title, Hire-Date, salary);
2. Department (Department-ID, Manager-ID).
3. Manager (Manager-ID, Name).

Examples

① Book database

Step 1:- Understand the tables and attributes

Author: Author-ID, Author Name, Author Country

Publisher: Publisher ID, Publisher Name, Publisher Address

Book : Book ID, Book Title, Author ID, Publisher ID, Price

customer : Customer ID, Customer Name, Customer

sales : Sale ID, Book ID, Customer ID, sale Date, Quantity, total Price

Step 2: Identify functional Dependencies (FDs)

- Author ID $\rightarrow$ Author Name, Author Country
- Publisher ID $\rightarrow$ Publisher Name, Publisher Address
- Book ID $\rightarrow$ Book Title, Author ID, Publisher ID, Price
- Customer ID $\rightarrow$ Customer Name, Customer Contact
- Sale ID $\rightarrow$ Book ID, Customer ID, sale Date, Quantity, total Price.

Step 5: Convert to BCNF

1. Ensure every determinant is a candidate key
2. check for overlapping candidate keys
3. Decompose relations to eliminate redundancy
 - No further decomposition needed

Using Griffith Tool

1. Input relational schema and functional dependencies
2. Griffith tool generates a dependency graph.
3. Analyze the graph to identify normalization issues
4. Apply normalization rules to transform the schema
5. Verify the resulting schema meets BCNF criteria

Griffith Tool Steps:

1. Create a new project in Griffith.
2. Define the relational schema and FDs
3. Run the "Dependency Graph" tool
4. Analyze the graph for normalization issues
5. Apply transformations using the "Normalize" tool
6. Verify BCNF compliance using the "BCNF Check" tool.



8
4
3
1
11
16

[illegible]

Result:- Thus the normalization of the given relation is created the simplified tables with suitable constraint successfully.

Step 3: Normalize Tables

1NF

All table already have atomic attributes.
Each field contains only one single value.

2NF

- 2NF requires 1NF and no partial dependency on a composite key.
- Since most tables have simple primary keys, 2NF is satisfied.
- If the Enrollment table uses (student ID, course ID) as a composite key, we must check for partial dependencies.

3NF

Remove transitive dependencies.

Look for transitive dependencies:

In course, Dept ID $\rightarrow$ Dept Name, Dept Head.

Instructor, Dept ID $\rightarrow$ Dept Name, Dept Head.

Hence, Department details should be stored separately.

Final Tables after 3NF:

1. Student (Student ID, student name, student Dept, student contact)
2. Course (course ID, course name, course credits, Dept ID)
3. Department (Dept ID, Dept Name, Dept Head)
4. Instructor (Instructor ID, Instructor name, Dept ID)
5. Enrollment (Enroll ID, student ID, course ID,

Final table after 3NF:

1. Authors (Author ID, Author Name, Author Country)
2. Publishers (Publisher ID, Publisher Name, Publisher Address)

Example!:-

② Student - Course database

Step 1:- Understand the tables and attributes

Student: Student ID, Student Name, student dept, student Contact

Course: Course ID, Course Name, Course Credits, Dep ID

Department: Dept ID, Dept Name, Dept ID

Enrollment: Enroll ID, Students ID, Course ID, Instructor ID, Grade Semester

Step 2:- Identify Functional Dependencies.

Student ID $\rightarrow$ Student Name, student Dept, Student Contact.

Course ID $\rightarrow$ Course Name, Course Credits, Dept ID

Dept ID $\rightarrow$ Dept name, Dept Head

Instructor ID $\rightarrow$ Instructor Name, Dept ID

Enroll ID $\rightarrow$ Student ID, Course ID, Instructor ID, Grade Semester.

Additional dependencies include:-

Course ID $\rightarrow$ Course Credits

(Student ID, Course ID) $\rightarrow$ Grade

(Course ID, Instructor ID) $\rightarrow$ Semester

- Alter Database
- Backup Server Parameters File
- Restore server Parameters File
- Backup control file
- Restore control file

VEL TECH	
EX NO.	
PERFORMANCE (5)	
RESULT AND ANALYSIS (5)	
VIVA VOCE (5)	
RECORD (5)	
TOTAL (20)	
DATE	

Result:- Thus, the Backing up and recovery in databases is verified and executed successfully.

Step 5: Apply Incremental Backup
Restore database [database-name] from disk
= 'Incremental-backup.bak' with Replace.

Step 6:- Recover Database
Recover database [database-name]

Step 7:- Open database

Alter database [database-name] set online

Scenario 2:-

Step 1:- Backup SPFILE

Backup Server parameter File To File =
'spfile.back';

Step 2:- Simulate Control File Loss
Delete or modify control file

Step 3:- Restore control File
Startup MOUNT

RESTORE control File from File = 'control file.
back'; Alter control file Resume;

Step 4:- Recover database

Recover database using Backup control file.

Step 5:- Open database

Alter database Open Reset logs;

SQL Server Commands:

- Backup database
- Restore Database
- Recover database

Task - 9

Backing up and recovery in databases:-

perform following backup and recovery scenarios

Aim:- To understand and perform various backup and recovery scenarios in databases

a) Recovering a No Archive log Database with Incremental Backups

b) Restoring the server Parameter file.

c) Performing Recovery with a Backup control file

Scenario 1:- Recovering a NOARCHIVELOG Database with Incremental Backups

Step 1:- Backup Database

Backup Database [database-name] To disk = 'backup-file.bak' with Noformat, No init, Name = 'Full Database backup'. skip, Rewind, No unload, stats=10

Step 2:- create Incremental Backup.

Backup database [database-name] To disk = 'incremental-backup.bak' with differential, Noformat, No init, Name = 'Incremental Database Backup', skip, Rewind, Download, stats=10

Step 3:- Simulate Data loss

- Intentionally delete or modify data

Step 4:- Restore Database

Restore Database [database-name] From disk = 'backup-file.bak' with Replace.

```
25 tags: ["blank", "red"], size: {h=14, w=21,
  uom="cm"}},
```

```
{item: "mat", qty: 85 tags: ["gray"], size: {h=27.9,
  w=35.5, uom="cm"}}, {item: "mouse pad", qty=
```

```
25 tags:
```

```
["gel", "blue"], size: {h=19, w=22.2, uom="cm"}},
  ]
```

```
{
  "acknowledged": true
```

```
  "inserted ids" = [
```

```
    object ID ("627d1598c73990c074e6397d")
```

```
    object ID ("627d1598c73990c074e6397e")
```

```
    object ID ("627d1598c73990c074e6397f")
  ]
```

```
}
```

```
> db.mylab.find({item: 1, qty: 1})
```

```
{ "_id": objectid("627d13acc73990c074e6397c")
```

```
  "item": "canvas", "qty": 100
```

```
{ "_id": objectid("627d1598c73990c074e6397d")
```

```
  "item": "journal", "qty": 25
```

```
{ "_id": objectid("627d1598c73990c074e6397e"),
```

```
  "item": "mat", "qty": 85
```

```
{ "_id": objectid("627d1598c73990c074e6397f"),
```

```
  "item": "mouse pad", "qty": 25
```

```
> db.mylab.find({item: 1, qty: 1}).
```

```
pretty()
```

```
{
```

```
  "_id": objectid("627d13acc73990c074e6397c")
```

```
  "item": "canvas
```

```
  "qty": 100
```

```
}
```


to confirm that your path environment variable is correctly configured to find mongosh, open a Command Prompt enter the mongosh-help command. If your Path is configured correctly, a list of valid commands displays.

step 4: open mongo shell 4.0 from C:\program files\mongodb\server\bin\mongo -exec.

step 5: TYPE the CRUD (CREATE READ UPDATE DELETE) commands given in text file

CRUD OPERATIONS:

```
db.createCollection("my lab")
{ "ok" : 1 }
```

```
{ db.my lab.insert one({ item: "Canvas", qty: 100,
tags: ["cotton"], size: { h: 28, w: 35.5, uom:
"cm" } }) }
```

```
{
  "acknowledged" : true,
  "inserted id" : ObjectId("627d13acc7399c07
4e6399e")
}
```

```
> db.my lab.find({ item: "Canvas" })
{ "_id" : ObjectId("627d13acc7399c074e63
97c"), "Item": "Canvas", "qty": 100,
"tags": ["cotton"], "size": { "h": 28, w: 35.5
"uom": "cm" } }
```

```
> db.my lab.insert many([ { item: "journal", qty:
```


CRUD OPERATIONS IN DOCUMENT DATABASES

Aim:- To Perform Mongoose using npm design on MongoDB designing document database and performing CRUD operations like creating, inserting, querying, finding and removing operations.

Steps:-

Step 1: Install Mango db using following link
<https://www.mongodb.com/try/download/community>

Step 2: Install mangodb using the below link
<https://www.mongodb.com/docs/mongodb-shell/#download-and-install-mangosh>

Step 3: To add the mongoDB shell binary's location to your Path Environment variable, open the control Panel

In the System and Security category, click System, click Advanced system settings. The system properties modal displays

click environment variables

In the system variables section, select Path and click Edit environment variable modal displays click New and add the filepath to your mongo-sh binary.

click OK to confirm your changes, on each other modal, click OK to confirm your changes.

CRUD OPERATIONS IN GRAPH DATABASES

Aim: To perform CRUD operations like creating, inserting, querying, finding, deleting operations on graph spaces

* Create Node with Properties

Properties are the key-value pairs using which a node stores data. You can create a node with properties separated by commas within the flower braces "{}".

Syntax:-

Following is the syntax to create a node with properties `create (node: label {key: value, key2: value, ...})`

* Returning the Created node

To verify the creation of the node, type and execute the following query in the dollar prompt.

`match (n) return n`

Creating Relationships.

We can create a relationship using the create clause. We will specify relationship within the square braces "[]" depending on the direction of the relation-ship it is placed b/w hyphen "-" and arrow "→" as shown in the following syntax.

"_id": objectId ("627d1597c73990c074e6397f"),
"item": "mouse pad", "qty": 254

VEL TECH	
EX NO.	
PERFORMANCE (5)	60
RESULT AND ANALYSIS (5)	4
VIVA VOCE (5)	4
RECORD (5)	3
TOTAL (20)	71
DATE	

Result: The implementation of CRUD operations like creating, inserting and removing & finding operations using MangoDB ~~is su~~ is successfully executed.


```
{
  "_id": objectId("627d1598c73990c074e6397d"),
  "item": "journal",
  "qty": 25
}
```

```
{
  "_id": objectId("627d1598c73990c074e6397e"),
  "item": "mat", "qty": 85
}
```

```
{
  "_id": objectId("627d13acc73990c074e6397c"),
  "item": "canvas",
  "qty": 100
}
```

```
"tags": [
  "cotton"
],
```

```
"size" = { "h": 28, "w": 35.5, "40m": "cm" }
```

```
zdb.mylab.deleteOne({ item: "journal" })
```

```
zdb.mylab.find({}, { item: 1, qty: 1 }).pretty()
```

```
{
  "_id": objectId("627d13acc73990c074e6397c"),
  "item": "canvas",
  "qty": 100
}
```

```
{
  "_id": objectId("627d1598c73990c074e6397d"),
  "item": "journal",
  "qty": 25
}
```

```
{
  "_id": objectId("627d1598c73990c074e6397e"),
  "item": "mat", "qty": 85
}
```

```
{
  "_id": objectId("627d1598c73990c074e6397e"),
  "item": "mat", "qty": 85
}
```

EX NO.	11
PERFORMANCE (5)	4
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	3
RECORD (5)	1
TOTAL (20)	11
SIGN WITH DATE	12

For

Result:- Thus implementation of CRUD operations in graph database using ~~MangoDB~~ is successfully executed

create a graph database for student course
registration, create student and dept node
and insert values of properties
create (n:Student {sid: "VTU14500",
sname: "John",
dept name: "CSE"})

Syntax:-

Following is the syntax to create a relationship using the create clause

create (node1) - [: Relationship Type] -> (node2)
• creating a Relationship b/w the existing nodes.
you can also create a relationship b/w the existing nodes using the HATCH clause
syntax:-

Following is the syntax to create a relationship using the HATCH clause

HATCH (a: label of Node1), (b: label of Node2)
WHERE a.name = "name of node1" AND
b.name = "name of node2"

create (a) - [: Relation] -> (b)

Return a, b

* Deleting a Particular Node:-

• to delete a Particular Node, you need to specify the details of the node in the place of "n" in the above query.

Following is the syntax to delete a Particular Node from Neo4j using the delete clause.

HATCH (node: label { properties ... })

DETACH DELETE node

create a graph database for student course